Welcome to Presently

A web-based presentation tool built with Lively

Take a breath and wait for the room to settle.

Hi everyone, thanks for being here. My name is Samuel, and today I want to show you Presently — a web-based presentation tool I built on top of Lively.

Make eye contact with the front row.

It's designed to make giving talks feel a little less stressful, and a little more in control.

The best way to predict the future is to create it.

未来を予測する最善の方法は、それを創ることである。

Read the quote aloud slowly, then pause.

That's a quote often attributed to Peter Drucker. I think it captures something important — the tools we choose to build shape the work we're able to do.

Presently has a few key features. Let's walk through them.

The foundation is real-time sync — the display and presenter views both connect to the server over a WebSocket, so when you advance a slide, the audience sees it immediately. No refresh, no polling.

- Real-time synchronization between display and presenter

Slides are plain text files. You write Markdown, add a little YAML at the top to set the template and timing, and that's your whole slide. No GUI, no proprietary format.

- Real-time synchronization between display and presenter
- Markdown-based slides with YAML frontmatter

There are several built-in layouts — title, statement, code, two-column, image, and the default bullet list you're looking at now. Each one is an XRB template, so they're easy to customise or add to.

- Real-time synchronization between display and presenter
- Markdown-based slides with YAML frontmatter
- Multiple templates for different slide layouts

This is the presenter view you're looking at right now. Each slide has its own notes — what to say, what to do — and a timer that tracks how you're pacing against the expected duration.

- Real-time synchronization between display and presenter
- Markdown-based slides with YAML frontmatter
- Multiple templates for different slide layouts
- Presenter notes and timing information

And finally, transitions. These are handled with the View Transitions API and standard CSS — no JavaScript animation library required. Morph, fade, slide left and right are all built in, and you can add your own with a few lines of CSS.

Open two browser windows side by side to demonstrate.

- Real-time synchronization between display and presenter
- Markdown-based slides with YAML frontmatter
- Multiple templates for different slide layouts
- Presenter notes and timing information
- HTML5 animations for smooth transitions

You are here →

Pause — let the callout land.

And yes — this slide is itself a live example. The "You are here" badge just faded up on the display using exactly the same animation system we're about to look at.

That's the whole point of Presently: the tool eats its own cooking.

Initialization

これは Presently の核心部分です — `Presentation` クラス。コンストラクタはスライドのディレクトリを受け取り、初期状態を設定してから `load_slides!` を呼び出してディスク上の Markdown ファイルをすべて読み込みます。タイミングを追跡する `Clock` が作成され、リスナー配列がオブザーバーパターンのために用意されます。

Highlighted lines 2–10 should be visible — check the display before speaking.

This is the heart of Presently — the Presentation class. The constructor takes a slides directory, sets up the initial state, and then calls load_slides! to read all the Markdown files off disk. A Clock is created to track timing, and a listeners array is set up for the observer pattern we'll look at in a moment.

Navigation

Slide 10 of 17 `050-code-navigation.md` Elapsed: 04:00 Slide: 01:00 Samuel

Navigation is handled by advance! and retreat!, which both delegate to go_to. That method does the bounds checking — so you can't advance past the last slide or retreat before the first.

Point to the go_to method on the display screen.

The key line is at the end of go_to: notify_listeners!. Any object that has registered itself — the WebSocket connection, the timer, the presenter view — gets called immediately when the slide changes. That's what keeps everything in sync with no polling.

An HTML grid layout with animated step-by-step reveals using `slide.after()`.

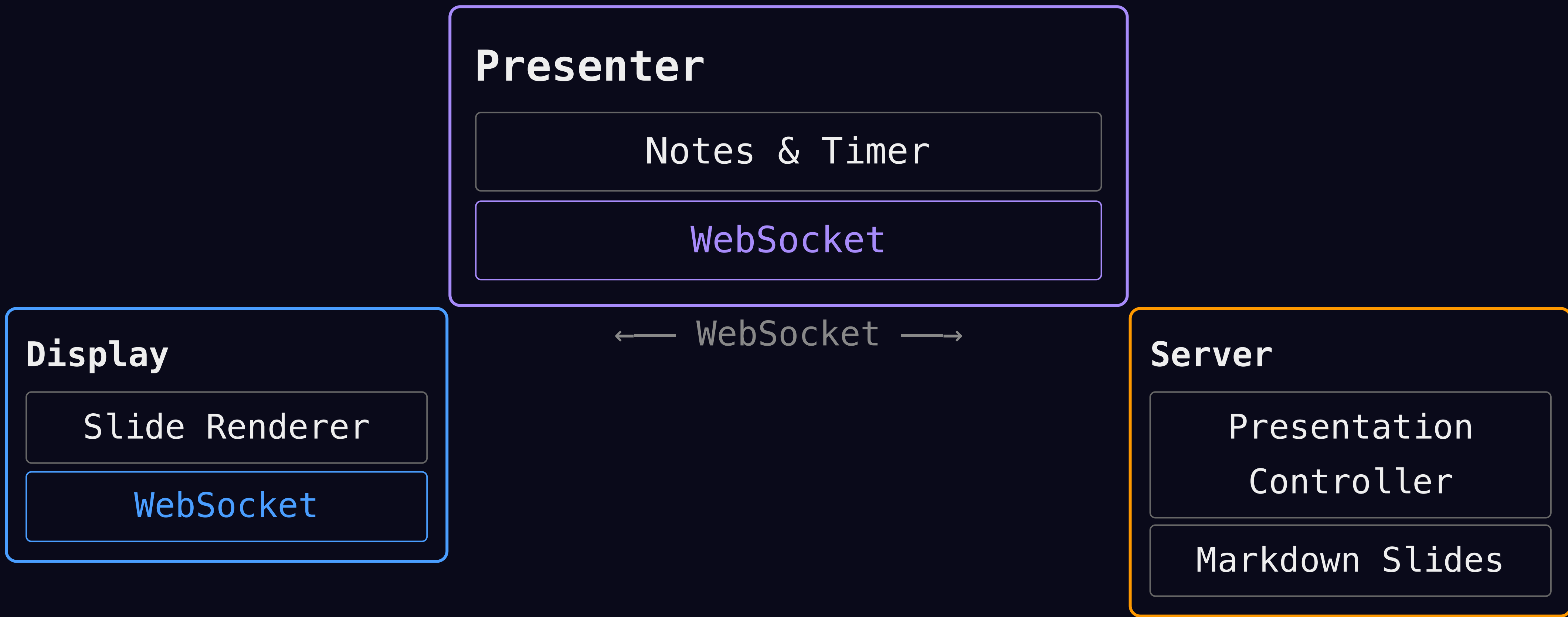
Server

Presentation Controller

Markdown Slides

Start with the Server box centred and large. Details build in one by one.

Server morphs to the right. Display fades in on the left. Connection label appears.



Display and Server morph smaller and move up. Presenter fades in at the bottom with details building in.

Request Lifecycle

① Client sends HTTP request

② Server parses & routes request

③ Presentation controller updates state

④ Listeners notified, slide rendered

⑤ Client receives update via WebSocket

A looping animation that replays the request lifecycle automatically using `slide.loop()`.

Demo Time

Slide 16 of 17 `070-section.md` Elapsed: 08:30 Slide: 00:15 Samuel

Switch to the demo browser tab.

Let's see it running live. I'll show you the presenter and display views side by side, advance through a few slides, and you can watch the sync happen in real time.

Return to the slides when done.

Thank You!

Questions?

Slide 17 of 17 `080-thanks.md` Elapsed: 08:45 Slide: 00:30 🎤 Samuel

Thank you for your time. Presently is open source — you can find it on GitHub under socketry. If you want to build your own tools for giving talks, I hope this gives you a starting point.

I'm happy to take any questions.

Advance the slide clicker to blank if possible, or just stand still.